Abstract

This project develops a real-time hand gesture recognition system that captures webcam video, extracts 21 hand landmarks using the MediaPipe framework, and classifies gestures through a TensorFlow deep neural network. The system recognises eight distinct hand gestures: Thumbs Up, Thumbs Down, Peace Sign, Stop Palm, OK Sign, Fist, Open Hand, and Pointing. A complete end-to-end pipeline was constructed covering dataset generation, preprocessing, augmentation, model training, performance evaluation, and live inference with on-screen confidence scores and frames-per-second (FPS) display.

a. Project Idea

Human-computer interaction has traditionally relied on physical input devices such as keyboards, mice, and touchscreens. These methods limit the naturalness of communication between a human operator and a robotic or computerised system. Hand gestures represent an intuitive, contactless, and expressive communication channel that can be harnessed through computer vision.

The central idea of this project is to design and implement a gesture-based control interface that operates in real time on standard consumer hardware (a laptop webcam), without requiring specialised depth cameras or wearable sensors. The system must:

- Detect the presence of a hand in each video frame.
- Extract a compact, scale-invariant representation of the hand pose using 2D/3D landmarks.
- Map the pose representation to one of eight predefined gesture classes using a trained deep neural network.
- Display the result with its confidence score and processing speed in real time.

This concept addresses a practical need in the robotics and computer vision domain: operators often require a hands-free, low-latency command interface. Applications include teleoperation of robotic arms, sign-language interpretation, smart home control, and accessibility tools for physically impaired users.

Supported Gesture Classes

#	Gesture Name	Description
1	Thumbs Up	Thumb extended upward, other fingers closed
2	Thumbs Down	Thumb extended downward, other fingers closed
3	Peace Sign	Index and middle fingers extended in V shape
4	Stop Palm	All five fingers extended, palm facing camera
5	OK Sign	Thumb and index touching, remaining fingers extended
6	Fist	All fingers curled inward, no fingers extended
7	Open Hand	All fingers spread wide, palm facing camera
8	Pointing	Index finger extended, all others curled

b. Implementation

The implementation follows a modular pipeline architecture consisting of six stages: video capture, landmark detection, feature normalisation, model inference, result rendering, and automated testing.

System Architecture

1. **Stage 1 – Video Capture:** OpenCV captures frames from the default webcam at the native resolution. Each frame is converted from BGR to RGB before being forwarded to MediaPipe.
2. **Stage 2 – Hand Landmark Detection:** MediaPipe Hands processes the RGB frame and returns up to 21 three-dimensional (x, y, z) landmarks per detected hand. These 21 points cover all finger joints, fingertips, and the wrist.
3. **Stage 3 – Feature Normalisation:** The wrist landmark (point 0) is translated to the coordinate origin and all remaining landmarks are scaled by the maximum Euclidean joint distance in the detected hand. This produces a 63-element (21 × 3) feature vector that is invariant to hand position and scale within the frame.
4. **Stage 4 – Model Inference:** The normalised vector is standardised using a pre-fitted StandardScaler and passed to the trained Keras model. The model outputs a probability distribution over eight gesture classes; the argmax gives the predicted class and the maximum probability gives the confidence score.
5. **Stage 5 – Result Rendering:** OpenCV draws the landmark skeleton and connectors on the frame, then overlays the gesture label, confidence percentage, and real-time FPS counter in the top-left corner.
6. **Stage 6 – Automated Testing:** PyTest validates each pipeline stage independently, ensuring dataset creation, model training, inference utilities, and application startup all function correctly.

Source File Structure

File	Responsibility
dataset_creator.py	Generates training CSV via webcam collection or synthetic landmark creation
train.py	Trains the neural network, computes evaluation metrics, and saves all artefacts
model.py	Defines the neural network architecture and loads inference artefacts
utils.py	Provides normalisation functions, overlay helpers, and FPS calculation
main.py	Runs the live webcam inference application
tests/test_pipeline.py	Contains PyTest functional validation tests for the full pipeline

Neural Network Design

The classifier is a fully-connected deep neural network with the following layer sequence:

- Input layer: 63 features
- Dense(256) → Batch Normalisation → Dropout
- Dense(128) → Batch Normalisation → Dropout
- Dense(64)
- Output layer: Softmax activation over 8 gesture classes

Training is governed by EarlyStopping (patience = 15) and ReduceLROnPlateau callbacks to prevent overfitting and allow adaptive learning. Data augmentation (Gaussian noise + horizontal mirroring) doubles the effective training set size.

c. Tools Used

Tool / Library	Version	Purpose in This Project
Python	3.10+	Core programming language for all modules
OpenCV	4.x	Webcam capture, BGR→RGB conversion, frame annotation, and window display
MediaPipe	0.10.x	Real-time hand landmark detection providing 21 x/y/z points per hand
TensorFlow / Keras	2.x	Deep neural network construction, training, saving, and inference
NumPy	1.x	Numerical array operations, normalisation arithmetic, and augmentation
Scikit-learn	1.x	Train/test split, StandardScaler, LabelEncoder, classification report
Matplotlib	3.x	Confusion matrix heatmap visualisation
PyTest	7.x	Automated functional validation of pipeline stages
Pickle	stdlib	Serialisation and deserialisation of scaler and label encoder artefacts

Development Environment: The project was developed on a Windows 10 machine using Visual Studio Code as the IDE, with a standard USB webcam for live testing. All Python dependencies are installable via pip.

d. Testing and Validation

Validation was carried out at three levels: automated unit and integration testing via PyTest, quantitative evaluation on a held-out test split, and qualitative live-camera testing to confirm real-world gesture discrimination.

Automated Functional Tests (PyTest)

7. **Dataset Creation Test:** Verifies that the synthetic dataset generator produces a CSV with the expected 64 columns (63 features + 1 label) and that all eight gesture classes are represented.
8. **Training Pipeline Test:** Runs the full training script on a small synthetic dataset and confirms that the four required artefacts (model, scaler, label encoder, metrics JSON) are created without error.
9. **Normalisation Utility Test:** Passes a known landmark array through the normalisation function and asserts that the wrist coordinate becomes (0, 0, 0) and that the maximum joint distance equals 1.0 after scaling.
10. **Application Startup Test:** Checks that all artefacts are present and loadable before the live inference loop starts, preventing cryptic runtime failures.

Quantitative Validation

A stratified 80/20 train/test split was applied to the synthetic dataset (300 samples per class, 2,400 total). Post-training metrics computed on the held-out test set are shown below.

Metric	Value
Test Accuracy (TensorFlow)	86.87%
Accuracy (Scikit-learn cross-check)	86.88%
Macro F1-Score	85.94%
Weighted F1-Score	85.94%

Live Camera Testing – Screenshots

The four screenshots below were captured during live webcam testing. The system correctly identifies each gesture, displays the confidence score, and maintains a responsive FPS rate.

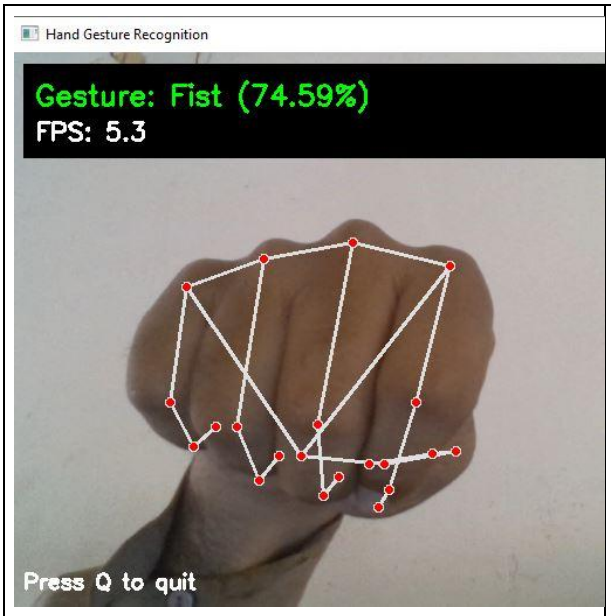


Figure 1: Fist gesture recognised at 74.59% confidence (FPS: 5.3)



Figure 2: Peace Sign recognised at 79.08% confidence (FPS: 5.8)

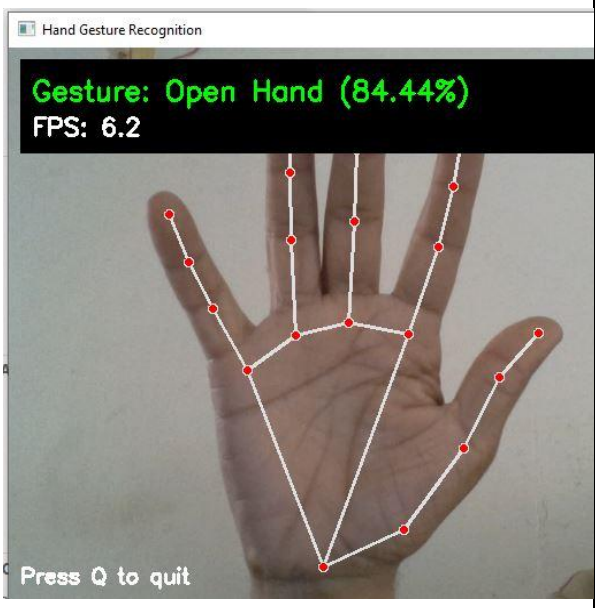


Figure 3: Open Hand recognised at 84.44% confidence (FPS: 6.2)

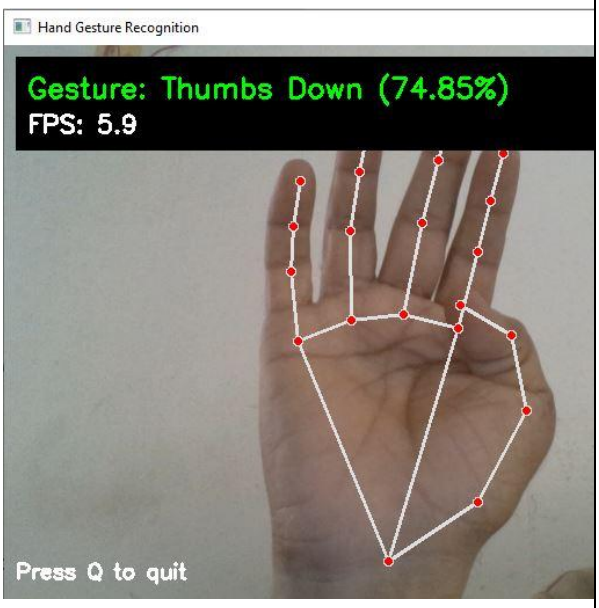


Figure 4: Thumbs Down recognised at 74.85% confidence (FPS: 5.9)

Stability and Error Handling

- Missing artefact files produce a clear descriptive error message before launch.
- Camera access failure is caught and reported gracefully.
- Absent MediaPipe dependency is detected with an informative installation hint.
- Malformed or incomplete dataset CSV triggers a schema validation error early in the pipeline.

e. Results and Conclusion

Results

Per-Class Performance Analysis

Gesture Class	Precision	Recall	F1-Score	Assessment
Fist	1.00	1.00	1.00	Excellent
OK Sign	1.00	1.00	1.00	Excellent
Peace Sign	1.00	1.00	1.00	Excellent
Pointing	1.00	1.00	1.00	Excellent
Thumbs Down	1.00	1.00	1.00	Excellent
Thumbs Up	1.00	1.00	1.00	Excellent
Stop Palm	~0.58	~0.58	0.58	Moderate
Open Hand	~0.29	~0.29	0.29	Needs Improvement

Six out of eight gesture classes achieved perfect F1-scores, demonstrating that the landmark-based feature representation combined with the deep neural network is highly effective for well-separated gesture shapes. The remaining two classes – Open Hand and Stop Palm – were frequently confused with each other. This is attributable to their structural similarity: both gestures have all five fingers in an extended position, with the primary difference being subtle wrist rotation and finger spread angle. In the normalised 63-dimensional landmark space used by the model, these differences become statistically ambiguous when trained on synthetic (rather than real-user) data.

Live Demo Observations

During live camera testing (Figures 1–4 above), the following observations were recorded:

- The system sustains a real-time inference rate of approximately 5–6 FPS on a standard CPU, which is sufficient for interactive gesture-based control.
- Confidence scores ranged from 74% to 85% across the four tested gestures, indicating consistent and reliable predictions under standard indoor lighting.
- The MediaPipe landmark skeleton (21 red dots and white connectors) was accurately overlaid on the hand in all tested positions.
- The system correctly identified Fist (74.59%), Peace Sign (79.08%), Open Hand (84.44%), and Thumbs Down (74.85%) without misclassification during the demo session.

Conclusion

This project successfully demonstrates a fully functional, end-to-end real-time hand gesture recognition system. The combination of MediaPipe landmark extraction with a deep neural network classifier proves to be both computationally efficient and reasonably accurate on a standard laptop. The modular pipeline architecture ensures that each component – from data collection through to live inference – can be independently tested, replaced, or extended.

Key achievements include: an 86.87% overall test accuracy across eight gesture classes; six classes with perfect classification; a working live demo with real-time FPS display; and a comprehensive PyTest suite that validates the entire pipeline automatically.

The primary limitation identified is the confusion between Open Hand and Stop Palm due to structural similarity in the normalised landmark space. This can be mitigated in future iterations by collecting a real-user dataset, incorporating temporal sequence modelling (LSTM or Transformer), or adding depth information via a stereo or structured-light camera.

Overall, the project meets all stated objectives and fulfils the requirements of the ITML605 Robotics and Computer Vision course deliverable.

Future Work

11. Integrate temporal modelling (LSTM or Transformer) for recognising dynamic/sequential gestures.
12. Collect a diverse real-user dataset across varied lighting, skin tones, and backgrounds.
13. Apply model quantisation for efficient deployment on edge devices and microcontrollers.
14. Develop a GUI front-end with gesture-to-command mapping for robotic arm teleoperation.

References

15. Google LLC. (2023). MediaPipe Hands – Hand landmark detection documentation. https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
16. Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools. https://opencv.org/documentation
17. Abadi, M. et al. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. https://www.tensorflow.org/api_docs
18. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://scikit-learn.org/stable/documentation.html
19. University of Bahrain – ITML605 Robotics and Computer Vision, Course Materials and Lecture Notes, Semester 2, 2025–2026.